

ActInf MathStream 013.1 ~ Alexis Toumi: "DisCoPy: Monoidal Categories f

MathStream | Alexis Toumi | 1:06:48

Hello, welcome everyone. It is May 20th, 2025. We're in ACTIMPH Math Stream number 13.1 with Alexi Toomey discussing DiscoPy, monoidal categories for active inference.

This should be a very exciting and useful presentation and stream. So thank you very much for joining and looking forward to learning more.

Yeah, thanks a lot. Thanks a lot for the invite. And yeah, I guess, as I was saying just a couple of minutes ago,

I'm very much looking forward to this talk. And it's a bit of a challenge for me, I guess, in a sense that I'm no expert in active inference.

So I'll do my best at explaining what I think active inference is about. And I also try to, yeah, convince the audience here that

DiscoPy, this Python library for string diagrams, is a promising way to experiment with active inference.

So I guess most of the references I'll be using for today would be this main DiscoPy paper.

You can also check out the documentation on GitHub. It's quite extensive. We've got plenty of examples and notebooks to play around with.

And then I'll give a small glimpse of what the DiscoPy software is used for. So this idea of QNLP, quantum natural language processing,

which is what motivated the library in the first place, what I wrote my PhD thesis about.

And I guess I'll give some introduction to category theory, which is the sort of mathematical foundations on which DiscoPy is built.

And then I'll be using mostly this recent paper from Toh, Kleiner, and Smyth on active inference in string diagrams,

where essentially they use the same mathematics that I was using for quantum NLP and apply that to active inference.

And in a sense, that's where I learned all I know about active inference. So I'll be giving a summary of the sort of theoretical background.

And then I'll be mostly showing some screenshots from that paper to try and show how that theory applies to active inference in particular.

And then we can discuss how DiscoPy could be used to implement active inference experiments.

So yeah, let's get going.

So I guess the main theoretical concept in DiscoPy is a box.

So like we draw these boxes a lot.

And I guess I don't roast coffee every morning, but I grind coffee, I brew coffee every morning.

And I guess that's something that you can understand in terms of a black box where you put a bunch of inputs like green coffee beans,

electricity, roast level, start signal.

It could be physical, it could be information.

And these boxes are sort of abstract conceptual operations that just have a name, a designated list of inputs

and outputs.

So you've got two sets, the objects, so the things that are on the wires and the boxes have names too.

And then you have these two maps from boxes to lists of wires.

So that gives you all the processes that you can talk about in your theory.

And then you can translate one theory into another using these morphisms of signatures.

So first, if you have a box from X to Y , you draw it with an arrow in this way.

And then if you have morphism of signature, what it means is you can translate wires of one into lists of wires in the second.

And then you can translate a box in one into a box in the appropriate thing in the other.

So if you have, so this should be Σ prime, I guess it's a typo, but you translate boxes in a way that respects the inputs and outputs.

And so that's very useful because if you have different kinds of processes and you want to translate between them, that concept of morphism of signature is going to be the key one.

And so I guess with that, you can define the set of diagrams.

So that's going to be the main, the main definition for this thought.

And you start with a signature and then you define diagrams by induction, where every box is a diagram.

The identity is a diagram and you draw it with just a bunch of parallel wires.

The composition of a diagram from X to Y and Y to Z is also a diagram.

So you can have two diagrams, you can plug them one after the other.

And then the tensor of two diagrams is also a diagram.

So if you have F and F prime, you can put them side by side and you get something that goes from XX prime to Y , Y prime.

And that's it. So you can put diagrams in sequence and in parallel and you can also do nothing.

So this identity diagram.

And then I guess the subtlety that we'll use here is that, well, I guess first you need to put some axioms on these.

So I guess you want tensor and composition to be associative.

You want them to be unital.

So if you compose with the identity, it doesn't do anything.

If you tensor with the empty diagram, so the identity on nothing on the empty list, that's also doing nothing.

And then you want just these two equations that tell you that you can switch identity and tensor.

So ID of tensor or tensor of ID is ID of tensor.

And similarly for the composition, the tensor of two compositions is going to be the composition of the tensors.

And we'll see what that looks like in terms of diagrams in a second.

But as I was saying, there's this sort of nice that you get is that if you take a signature and you look at all the diagrams that you generate from it, that makes a new signature.

And you can use that as a signature where essentially now the boxes have diagrams inside them.

And so that's a way of formalizing an idea of hierarchical diagrams where essentially a sub diagram will

contain or a sub diagram is a box that contains a diagram inside it.

So we'll use a lot of these hierarchical diagrams where we'll see the boxes serve to encode normalization.

And so, yeah, to get back to this main equation, I guess that's the only equation you need to remember.

And you don't actually need to remember it because essentially you got it built into your visual system.

So that's the sort of main argument for why you should use string diagrams is that essentially they encode this very natural way of looking at processes as things on the plane,

where the axioms become just manipulation of these diagrams geometrically.

So for example, this axiom of naturality where you exchange the composition and the tensor, this amounts to having either the boxes composed first sequentially and then in sequence or first in sequence and then sequentially.

And that gives you the same diagrams in the end.

So the only thing that counts is how things are connected, I guess, is the main idea for that axiom.

And so you can apply that to cooking.

The ingredients are going to be your wires.

The steps, the cooking steps are going to be your boxes.

And now anything that you can compose with these cooking steps will be a recipe.

It takes a bunch of inputs like eggs and gives you a bunch of outputs like whites and yolks.

Right.

And yeah, I guess this this axiom of exchanging tensor and composition essentially it amounts to formalizing what we mean by independence.

So if we have two boxes that are not connected like tracking here and parking here, we can perform them in any order.

If it doesn't matter how we decompose this tensor of composition.

So yeah, I guess let's look at what this diagram looks like in Python.

So as the main feature of this copay is you get this very nice syntax for expressing any such string diagram as essentially as a Python function.

So you start by declaring some types for the ingredients of your recipe like egg, white and yolk.

You have a box for cracking an egg and giving white and yolk.

You have these parameterized boxes.

So merge, you can merge anything.

So any X and that takes two X 's and gives you an X .

And now to define the diagram on the right, you'll just define a Python function, declare its inputs and its outputs.

And now if you take X and Y for the two eggs, you crack them, you get a B is a white yolk pair.

CD is a white yolk pair.

So you can merge the whites together, merge the yolks together and return.

So you get your diagram and you can draw it.

So this output on the right is actually what this copay gives you as a drawing.

And you can also write it in terms of purely pure operations of tensor and composition.

So this at symbol is how you do the tensor, the parallel composition.

And these chevrons do the sequential composition.

And here you notice that we need this special box for the swap, right?

Like at first with just tensor and composition, you can only do planar processes.

So here you need this special swap box that takes yolk and white and gives you white and yolk.

So yeah, I guess that's the sort of simplest example of how to use this copay is you can define a bunch of types, define a composite model, composite diagram out of these basic steps, basic boxes and types.

And then you can draw the results.

I guess drawing the result would be the first step.

And in a second, we'll look at what else you can do with diagrams.

So a more interesting example, I guess, in our context will be a generative model as a diagram.

So on the left, you have the sort of basic example of a probabilistic model where you have random variables numbered x_1 to x_5 .

And then you have these arrows.

I guess here some arrows are designated as sort of input arrows.

Here on the left, x_2 and x_3 are sort of variables that I guess you get to control or you can choose some distributions over these.

Then you have some hidden variables like this x_1 , x_2 and x_4 .

And you have observed variables, right?

The x_5 and x_3 would be the things that you can actually observe in the end.

And then sort of implicitly in every such probabilistic model or graphical model, you would have conditional probability distribution for every variable given the variables that point to it.

Right?

And so in the world of string diagrams, these conditional probability distributions become the important thing in a sense.

Like I guess here the labels, you could almost forget about these labels x_1 to x_5 .

They help to make the correspondence here.

But the actual important operations are these conditional distributions.

So x_1 , well, you need a distribution on it.

It's this latent variable.

That's going to be your a is going to be a prior on x_1 .

And then you need a distribution for b that gives you x_4 based on x_1 , x_2 .

You need distribution c that gives x_5 given x_4 , x_2 , x_3 and so on.

And then here you also need this important operation that does copying, right?

So here the variable x_2 will be used in the conditional distribution for x_4 and for x_5 .

So you'll feed it to b and you'll feed it to c .

So you need to copy it, right?

You need to broadcast this whatever input you feed in here, right?

And so if you take a distribution that's non-trivial, well, this will create some correlation between those

distributions, right?

And so at the end, what you get are the observed variables, which are these outputs of your diagram.

And so you get this whole mechanical translation where every probabilistic model can be faithfully encoded as such a string diagram.

The wires are the random variables.

I guess the sort of here is like the wires up to this copying, right?

So here the variable x_2 appears twice.

So you need to split the wire in some formal sense.

The boxes are conditional probability distributions that define your model.

And the diagrams are the generative models themselves, right?

So here's what the same code would look like in this code pipe.

Again, you're going to instantiate a bunch of types for x_1 to x_5 .

And then you're going to have these boxes, ABC, and you can define your model as a Python function that takes a bunch of variables and just does the usual thing.

And I guess the magic of these Python decorators is that your function here is not going to be interpreted as an actual function.

It's going to be executed symbolically.

And these variables x_2 and x_3 are going to be turned into the input wire of your diagram.

And then these x_5 , x_3 will be the output wires of your diagram.

And so in a sense, what this copay gives you is a sort of domain-specific language where you can write any such

generative model and you will get a data structure for it.

So I guess today I won't have much time to get into the data structure underlying these, but I guess there's sort of two main ones, two main ways of understanding these diagrams is the first one is as a sort of sequence of layers where each layer of your diagram is like, for example, this first layer, you've got a box A in parallel with copying, in parallel with another copying.

The second layer would be B in parallel with a bunch of identities and the last layer would be C.

So this sort of list-like representation is very convenient for drawing.

And I guess if we go back to our cooking example, this is why you get to see these sort of stacking of boxes is essentially it's more convenient to have one box at each layer.

So you just get a list of these guys, a list of operations that you perform.

And then there's sort of the graph-like representation, which is the more natural one when you think of a graphical model.

They're called graphical because it's a graph, right?

So I guess this copay gives you this ability of going back and forth between graph-like and sort of list-like.

And the list-like operation is also how you define the evaluation of these diagrams.

So we'll get to see that in a minute.

These diagrams, you can draw them, you can store them as combinatorial representations of graphs.

And I guess the main point is you can evaluate them and you can do experiments with them.

So let's get to that.

I guess the theory we need behind this is that of strict monoidal categories.

So I'll just say category for sure, because I guess they'll all be strict and monoidal in this talk.

And essentially, once you know what a signature is, so this set of wires and boxes with domain co-domain, you can define a category as a signature with a way of evaluating diagrams, essentially.

So a way of taking a composition of boxes and wires to give a single box.

So first you need a way of interpreting identity for each object.

You need a way of interpreting composition.

And you need a way of sequential and parallel composition, right?

So now essentially a category will be a concrete way of interpreting these diagrams.

And I guess the key sort of structural theorem for...

Well, okay, let's first look at some examples.

So I guess the main example we've seen so far of a category is the category of the wires and diagrams where the objects are these wires, the lists of inputs or outputs.

And the diagrams are this composition of boxes that you can represent combinatorially as graphs.

And then the...

I guess the most natural interpretation of these, if you come from some sort of mathematical background, would be to think of sets and functions, right?

So every box will be a function of its input wires into its output wires.

So you think of Cartesian product, or you get multiple inputs and you get multiple outputs.

I guess sets and relations are super interesting too, instead of...

So I guess in that case, you can do a lot of fancier diagrams if you think of relations.

So you don't have inputs and outputs anymore, but you can essentially think of them going both ways.

Then I guess, yeah.

The main example of these monoidal categories was to do linear algebra to sort of formalize things that physicists were already doing, drawing diagrams, linear maps as diagrams.

So I guess that's the main example.

And in our case, what we're interested in will be more something like the category of measurable spaces and stochastic maps.

So we'll think of each wire as a measurable space with the product space being the tensor, the operation that puts wires side by side.

And then a box will be essentially a stochastic map that takes a product space and gives you a new product space.

And so I guess with this interpretation in mind, essentially a graphical model, a generative model is precisely a diagram with an interpretation in stoch.

So I guess, yeah.

Then the main way to formulate this idea is essentially to define what's called the functor is going to be a morphism of categories.

So something that translates a signature into another and also preserves identity, composition, and tensor.

And then you have this sort of main theorem that tells you that diagrams are the free monoidal category.

So what this means intuitively is that if you want to define a functor from the category of diagrams, the only thing you need to do is to define an interpretation for each wire and interpretation for each box.

So on the left-hand side, you've got all the functions.

So on the left-hand side, you've got all the functors from C to D .

And here you've got all these sort of much smaller objects where essentially you just need measurable space for each of the wires and a stochastic map for each of the boxes and conditional probability.

And now you get an interpretation for each composition of these.

So that's sort of the structural theorem, the sort of foundational theorem that justifies why are string diagrams the right data structure when you're looking at composition in sequence and in parallel.

So any composition, any such composition is a diagram.

And so I guess with that theorem in mind, like on the right-hand side, you've got something that you can represent as a small data structure that is, you just need the representation for a stochastic map for each of the boxes in your diagram.

And on the left-hand side, you've got the sort of algorithm that's going to take these diagrams and compute their meaning by plugging these distributions together and essentially performing inference.

So on the right-hand side, you've got something that is sort of data-like.

On the left-hand side, you've got the algorithm that's going to compute that data to give you inference algorithms.

And I guess, yeah, I was promising some short detour into quantum natural language processing.

So I guess here it's just a small code snippet to show, I guess, how concise you can make a particular model if you write it as a functor.

So here we've got boxes are worlds. So we've got a box for Alice, a box for love and a box for Bob.

We're going to compose them together according to their grammatical structure.

So here the, the, the, the word Alice is connected to love via this cup and the word Bob is connected to love via that cup.

And overall you get a sentence.

So that's the sort of natural language side of things.

And then you get the quantum side of things by saying, look, every noun, I'm just going to interpret it as a qubit.

So as a quantum system that I can control.

The noun Alice is going to be one state of the qubit, the state zero, the word Bob is going to be the state one.

And now the world love is going to be where interesting stuff happens where I'm going to entangle or correlate my inputs and my outputs, my subject and my object.

So here you represent it as a quantum circuit.

That's going to be some, some hard amount gate and some controlled gate.

So essentially a bunch of quantum gates represented as boxes that you compose in sequence and in parallel.

So I won't get into much quantum theory here, but I guess the, the main punchline is that you can replace quantum by probabilistic circuits.

And the same theory applies in a sense that you get this compositional way of interpreting, well, natural language, but interpreting any sort of compositional system in general.

So here you take your sentence, you apply the functor to it, you will get a circuit, a quantum circuit.

And then essentially this eval is going to be a second step of the functor.

That's going to take your quantum circuit and turn it into a bunch of matrices.

So it's going to turn it into vector spaces and linear maps and compute the meaning of that circuit, compute the meaning of that sentence.

So I guess that's one QNLP experiment in, in a few lines of Python.

And I guess, yeah, the argument for this talk would be that we can replace quantum with, with active inference here and, and get a very similar recipe for how to build a larger active inference systems from smaller ones and then interaction.

So, yeah, I guess that's one of the, I guess, three years of PhD thesis in one slide.

So let's, let's move on.

Yeah, I guess I'll try and give some of the sort of necessary theory to, to look at active inference formally, but give it all in terms of diagrams rather than complicated formally.

So now that you've seen a bunch of diagrams.

You can, you've noticed that you, if you only have tensor in parallel and in sequence, you can only do planar diagrams.

If you want, if you want arbitrary diagrams, you need this special swap.

And the swap is going to just satisfy these four actions you see here.

If you swap with nothing, it doesn't do anything.

If you swap and swap back, it doesn't do anything.

If you swap F and G, it's the same as doing G and F and then swap.

And finally, you can decompose a complex swap in terms of basic swaps.

So that's all you need to get a symmetric monoidal category.

And essentially in a symmetric category, you don't care about the order of wires because you can permute them.

So in the case of probability theory, you don't care about the order of your probabilistic model.

You can permute the variables.

What matters is what's connected to what via boxes, not the order of the inputs and the outputs.

And yeah, I guess in natural language, you do care about the order of world.

So in that case, you are not symmetric.

But another kind of category once you have symmetric is the idea of a compact category.

And that's where you have the ability of bending wires.

So I guess in a normal string diagram, the wires are going to flow from the inputs to the outputs.

In a compact diagram, you can flow backward.

And in the first paper applying category theory to Bayesian inference, that was the technical apparatus to formalize the idea of Bayesian inverting.

So I guess with these cups and caps, you can take a joint distribution on A times B and bend one of the legs to get a stochastic map from A to B .

And similarly, if you have a stochastic map, you can bend the output and get a joint distribution.

And now if you have a stochastic map from A to B and you have a prior on A ,

so I guess that's the little trick that they had to hide is this little box, this little black box on the cup here.

Then you can invert.

You can perform Bayesian inversion and you will get a stochastic map from B to A .

And I guess, yeah, there are many ways of formalizing this caveat of you need a prior if you want to be able to do Bayesian inversion.

And yeah, I guess I won't get too much into the literature of how you formalize categories of stochastic processes are very big topic.

But in that context, having a compact category means essentially every object, every wire comes with a prior.

And now when you invert, you use that prior.

So, yeah, I guess in quantum, it's got a very different interpretation where these wires are going to represent Bell states, entangled states and entangled measurements.

And then this equation amounts to teleportation.

In the context of Bayesian probability, this equation is just the Bayesian inversion of a trivial probabilistic channel is itself.

So I guess not very interesting in quantum.

It's like, wow, we can teleport things in Bayesian.

It's just the inverse of the identity is the identity.

So that's, yeah, that's a compact category.

You can bend things around.

So if you have symmetric plus compact, you can truly talk about a graph structure for your diagrams because you can connect any input to any output in a graph like way.

So now I guess an important ingredient we've seen for defining a generative model is the ability to use the same variable in multiple places.

So that's formalized in terms of these co-monoids.

So you have an operation that splits a wire and it's co-commutative.

So if you split and then you swap, it's the same thing as just splitting.

It's got a unit which is discarding.

So if you split and then you, if you copy something and then you discard, or if you split and then you terminate the wire, you haven't done anything.

And it's co-associative.

So it doesn't matter if you, if you copy the left-hand copy or the right-hand copy, you get three identical copies.

And yeah, I guess the main two axioms for being a Cartesian category are these two.

So the second one, I guess you can call normalization.

So essentially in terms of stochastic maps, what this means is that if you apply a stochastic map and then

take the weight of the results,

take the mass of the results for any distribution coming in here, it's the same as the mass of the result, as the mass of the input.

So essentially, if you have a normal, it takes a normalized distribution to a normalized distribution.

And so on the left-hand side is more controversial, is not going to be true in the category of stochastic maps.

This says that F is deterministic. So if you perform F and then copy the results, it's the same as copying the inputs and performing F twice.

And I guess if you take the special case where F has no inputs, this means that essentially F can be copied, means that it's deterministic in a sense that performing F , if F was a TOS coin, right?

So you would take F through a coin and then copy the results.

You would get a different result as if you just two coins, right?

So the second one is going to hold in terms of Bayesian probability theory, if you have normalized processes.

And it's this idea of essentially you can also formalize it as causality.

So if you discard the output, it cannot influence the input.

And yeah, the first one is essentially what brings you back to the world of deterministic processes.

So it's a good way of characterizing which stochastic map is in fact a deterministic one.

And so if you have these two axioms, you're called the Cartesian category.

If you don't have them, you're called the copy-discard category.

And so I guess in our context, we are really interested in copy-discard categories where these two are different processes.

And yeah, I guess what can you prove at this abstract level of generality is a really nice no-cloning theorem where you don't need to assume any complex numbers or any quantum stuff.

And in fact, it holds as well in the context of probability theory.

And it's like, I think it's called the no-broadcasting theorem in that case.

But the idea is that you cannot have both cups and caps and copying.

So here, I guess you assume that you have cups.

And then because you have copying, then the copy of a cup is also going to be two cups, right?

But you can also do some shenanigans where you bend the wires because copying is co-commutative.

So on one side, you're going to swap the wires.

And now you've essentially proved that two cups like this side by side are actually equal to two cups nested inside out.

And then you bend all the legs and you get that a swap is equal to the identity, which is basically, yeah, it means you've done something wrong, right?

Like if you can't even distinguish swapping stuff from not doing anything, your category is very much trivial.

So I guess that's the main thing.

The no-cloning theorem is that if you're both compact, so you have cups and caps and copying, now you have to be a trivial category in the sense that everything is essentially a scalar multiple of the identity.

So that's not the case in probability theory.

So probability theory also has this no-cloning feature.

And that's essentially what you can't copy.

You can't copy something if it's a non-trivial distribution.

You can't copy a correlated state, right?

The same way you can't copy an entangled state in quantum.

So, yeah, I guess the last ingredient we need in order to talk about active inference is the idea of normalization.

And in the context of string diagrams, that's going to be represented, as I was saying, by hierarchical diagrams,

where you have a sub-diagram that's in dotted lines, will just represent the normalization of it with some caveat

if there's no support for the particular state.

So here you have a stochastic map from X to Y , and you put it inside of a dotted line, and you get its normalization.

So I guess the guy inside is not really a stochastic map anymore.

It's more of just any measurable function, not necessarily normalized.

And the thing outside becomes a proper stochastic map.

And that means for any measurable function, you can decompose it in terms of something that's normalized, and then something that you discount. So these three lines are the discounting operation that we drew as a white dot before.

But so essentially here, that equation tells you that every measurable function is going to be a normalized one up to some renormalization.

So I guess it's sort of intuitive. And also, normalization behaves nicely with respect to parallel composition.

So if you have f and g , and then parallel, you normalize them, you get the normalization of f ,

and so the normalization of g . It behaves well with respect to copying a particular variable of your generative model.

But the thing it doesn't behave well with is the composition.

So it's not true that the normalization of composition is the composition of normalization.

And that's essentially what gives you the difference between a Jeffrey update and a Pearl update in the context of Bayesian inference.

So on the left hand side, you've got a Jeffrey update, where you first normalize and then pre-compose with the prior.

And so I guess here on the left, if you want to obtain something normalized overall, you will need to renormalize.

So like you get two normalizations on the right hand side, you only get one.

And that's sort of the difference. That's the fact that normalization is not a functor.

In a sense, it doesn't commute with the composition.

So then another kind of bubble we'll need is taking the logarithm of a function.

So if we have a function from x to the reals, the positive reals, we can take its logarithm.

This function is just going to be if we think of the category of measurable functions.

Well, it's the empty diagram is going to correspond to the real numbers to the one dimensional space.

And so such a function will just be a box, an effect which takes something and gives you just a renormalization factor.

Right. And now if you have such an effect, you can put it into a green box and you'll get the the negative log of it.

So that's sort of a mere sort of notational gadget.

And essentially, yeah, it only becomes interesting if you start using it inside a more complex diagram where this this green box will not in will not commute past the composition and will give some interesting hierarchical structure to your to your diagram.

And I guess here I'm just copying some caveats from from the to paper.

And yeah, I guess they they give some some pointers to some interesting category theory that could be formalized to to explore these log boxes further.

But in our context where we need them to is essentially we yeah, we just want the nice properties of logarithms.

So if this copying operator acts like essentially a product, then we want that the log of a product is the sum of the logs.

So you get this sort of nice nice operation nice homomorphism property of the log.

The discarding operator acts like the unit of this multiplication.

So the log of the units will be zero and then you get this nice product rule.

The sort of lightness rule for the logarithm of of a tensor product in that sense will be the sum of two tensor products.

So that's also the kind of equation you'll find in a sort of automatic differentiation framework where the gradient of $D \otimes E$ is going to be the gradient of D plus the gradient of E .

And finally, if X is deterministic, so that's what they call sharp states, then you also commutes past the log.

So you get these nice property of the logarithm which allows to take a diagram with these green boxes and sort of massage them into a sum of diagrams.

And finally, you can define this notion of open vibrational free energy.

Instead of looking at the free energy of a closed model, so something that wouldn't have an input I here.

So just joint distribution of a state's internal states and observed variables.

So here's the formula.

So I guess that was sort of the main point of the paper was to build all this categorical approach.

So you have this idea of an open model.

So you have some of the inputs that you can control.

And now you can define the free energy of this model with respect to joint distribution on the inputs with respect to the internal state.

So you can put a prior on the observed variables.

So I guess that was sort of the main point of the paper was to build all this categorical apparatus to get to the point where you can just draw the definition of free energy as diagram.

of free energy as diagram.

And then the main theorem being that essentially,
this gives you a compositional definition
in a sense that the free energy for composition,
so it's the tensor composition,
the free energy of the tensor will be some composition
of the individual thing.

So I guess in the case of tensor is the simplest case,
I won't show the case for sequential composition
where you need to take a more complicated operation
to compose the subsystems.

But I guess that's sort of the main theorem
of that categorical active inference framework
is that you get this idea that the free energy
for the whole is composition of the free energy of its parts.

And essentially, yeah, mono-edal category theory gives you
the sort of the high level layer of abstraction
to describe these sorts of compositional phenomenon.

And yeah, I guess with this sort of foundational theory,
you want to turn it into application.

And so I guess that's what this copy was built for.

So what we already have to support this sort
of active inference implementation in terms of diagrams,
well, we have most of the finite dimensional case
implemented already in the sense that
if your random variable span categorical distribution,
is a categorical distribution,
and you can turn this Bayesian inference problem
into a tensor network contraction problem.

We have a bunch of automatic
and diagrammatic differentiation algorithms implemented
with some compilation with the JAX library.

So essentially, you can take your diagram
and compile it into a bunch of GPU code
for very fast execution.

And finally, the sort of more experimental,
more speculative part of the library
is this idea of the geometry of interaction,
where you formalize bi-directional processes.

So sadly, I won't have much time
to get into that side of things,
but there's this very interesting idea of a Bayesian lens,
where you have these bi-directional operations
that also have some stochastic side to them.

So essentially, you can talk about updates,
about the forward part of your process being some inference
and the backward part being some update.

And this geometry of interaction will give you a way
of composing these updates together
in a compositional structure of Bayesian inference.

So we already have the sort of purely abstract implementation
of this geometry of interaction.

What's missing is sort of plugging these two things together.

So just looking at how, if you take finite dimensional
as a first step, and you look at how it updates,
you should be able to implement it
in a purely compositional way with DiscoPy,
and also be able to draw the results in the end.

And I guess what's missing would be
a more continuous implementation,
where I guess to go beyond finite dimensional,
you need some sort of more involved implementation
based on monads, I guess.

It's a very well-developed theory on the category side.

And there's one thing we've started working on,
which is this idea of delayed interaction,
where essentially you can represent discrete time processes.

So they have a very natural way of talking about time series.

And yeah, I guess the main ingredient that's missing
to develop this library further
would be having experimentalists ready
to take the challenge to build their experiments
in a way that you can compose them
with other experiments in the future
and build it upon a theory
of how to compose processes together.

So I guess that was sort of the main objective of DiscoPy

was to sort of lower the entry to the cost of entry
to understanding string diagrams
so that any experimentalists from quantum computing
or active inference can pick up
just enough abstract mathematics
to turn these experiments into something
you can compose together.

And yeah, I guess I'd be happy
to get some questions from the audience
and looking forward to more interactions
with the active inference community.

Thanks a lot.

Thank you.

Great talk.

Okay.

Anyone watching live can write a question in a live chat
and I wrote down some questions as well.

So first though, just to kind of pull back,
like how did you get into writing DiscoPy?

What brought you to wanting to build this toolkit
and what made you interested in the area?

Yeah, I mean, so I guess it started
with the master's thesis in Oxford
was mostly quantum stuff
with the course that Bob Kirk was teaching there.

And I was interested in AI already
and NAP in particular at the time.

And I guess, yeah, there was this program
of bridging the two together
and making quantum NLP.

And so I got involved in that.

And when it came to implementing it,
it felt like it would be a shame
to implement quantum NLP
as a one-shot thing
where you build your experiment
and throw it away.

And I guess we took it as an excuse

to develop something
that we thought was much more broadly applicable
where you take this general theory
of mono-edal categories
and implement that
and use QNLP as a sort of proof of concept
that it can do something useful
and then hope that other people
will start formalizing more stuff
as mono-edal categories.

So I guess this paper from Tao et al
came after the thesis.

And I guess they took active inference
as something that you could formalize
with diagrams too.

So yeah, I guess QNLP was a bit of an excuse
to get into some very cool maths
and to develop some cool software
to back it up.

And now I'm sort of, yeah,
looking forward to seeing other ways
to apply this software.

Awesome.

Okay.

I'm going to read and answer one question
from the live chat
and then I'll ask you a question.

So Divital wrote,
is this under a specific group project
at Active Inference Institute
like Ontology or other project?

So just to briefly address that,
it could be super relevant for Ontology.

We could use the Active Inference Ontology
to extract natural language descriptions
of generative models, for example,
represent them in Discopy.

So it could be a tool used in Ontology Project

or it could be spun up
as a category theory project
as we've done in the past
when we, for example,
focused on some of Toby St. Clair Smith's work.

So everyone's welcome to get involved
and reactivate projects
as they are excited by it.

This is definitely something exciting.

So let me ask you this question.

How do we connect the properties of diagrams?

For example, the ability to swap,
capture causality in the era of time,
cups and caps and all that,
with the features of cognitive systems
which we're interested in?

like, how do we know which kinds of formal constraints
are overkill, non-sequitur?

How do we really like reconcile
and build with the right formal systems
when we're dealing with embodied cognitive systems
that don't, on a first pass,
have those kinds of constraints?

Yeah, I think that's a super interesting question
and in a sense,

like not being an expert in active inference,

I don't feel entitled to really say

what are the properties

that you guys are interested in.

But I guess as a sort of parallel

to other ways diagrams have been applied

in particularly in quantum,

I felt like one interesting way

to look at diagrams

is to look at them

in terms of combinatorial structures

that have some structural properties

that you can exploit

to turn the computational problems
associated with them
into tractable problems.

So I guess maybe as a bet,
I would say that characterizing
which models are efficiently simulatable,
simulatable or not,
it's sort of a key question
that you ask yourself
when you get to build a model
of cognition in general
in a sense that if you cannot simulate it,
if you cannot build
an efficient implementation for it,
then it's likely that the brain
isn't going to be working that way.
In a sense, the brain is efficiently
solving this cognition problem.

So behind it,
there must be some sort of
tractable algorithm.

And I guess there's a way
of characterizing which problems
will be tractable
by the shape of the diagrams
associated with them.

So one particular way this happens
is when you look at tensor networks,
if you can bound the width
of your diagram
and make it look like
it's almost a tree
and you have this idea
of bounded tree width
where you can show
that your network
has some sort of hierarchical structure to it
where it's shaped

in terms of a main root
that splits into branches.
Even if these branches are thick,
in a sense,
it's not necessarily a tree,
then this bounded tree width
will give you a bound
on how hard it will be
to compute this tensor network.
So in a sense,
from the sort of graph structure
of your diagram,
from the fact that this graph
has a nice decomposition,
you can exploit that fact
and turn it into an efficient algorithm
to solve inference on that model
or to contract the tensor network
in the case of physicists.
But I think this general principle
applies in a very broad range of cases
and there's some very deep intersections
with complexity theory
where I guess this P and NP question
is all about
how do you separate
between things
that you can solve efficiently or not.
So diagrams give you
an interesting way
of looking at this P versus NP question
where I guess you can try
and tell apart these two classes
by looking at the shape of diagrams
that represent these problems.
Awesome.
Okay.
So what makes the variational free energy

open or not open?

And then some more VFE-related questions.

How did you connect that

to the log properties

since, of course,

the free energy is used

as a bound on log probabilities

as surprise?

And then what does it mean

that the VFE is compositional

since from a numerical perspective,

the compositionality

is kind of trivial?

Like, of course,

I could calculate the free energy

for two different situations

and just add those numbers together.

So what do we really get

with the semantics of composition

that go beyond just calculating two numbers

and then adding those two numbers?

Yeah.

So I guess to go back

to that definition

of open free energy,

I guess the main difference here

compared to the standard definition

is this extra wire I, right?

So I guess

the idea is that

if this wire is empty,

if there's no particular input,

you get back to the usual definition

and it's the idea of

instead of trying to compute

the free energy

of a big monolithic system

that represents the overall thing

you want to study,
you first decompose that
into a composition
of different things
being independent
and being composed in sequence.
so that's where you get
the correlation
and this decomposition
of your problem
will give you a formula
to compute
the composition
of the corresponding
free energies.

And I guess
this example
wasn't necessarily
the best one to pick.
So here it's the fact
that it's compositional
with respect
to independent
parallel composition.

Well, I guess, yeah,
it's a bit of a trivial property
of free energy
that the free energy
of the tensor
of two things
will be the sum
of the free energies.

And I guess, yeah,
I should have included
the more interesting case
where you have
a sequential composition
of things

and where this sequential composition
gives you a non-trivial way
of decomposing the formula
for your free energy.

So I guess that would be
the place where you can get
some traction
in a particular experiment
where essentially
you can, yeah,

I guess the same way
that an automatic
differentiation framework
would use the structure
of your program
to compute the structure
of its derivative.

Well, here you could have,
yeah, I guess,
automatic free energy idea
of the structure
of your model
informs the structure
of the formula
for its free energy
and potentially implying
some algorithmic speedup.

So I guess here,
obviously,
there isn't any speedup
to be found
in this particular
instance of compositionality.

and, yeah,

I guess I'm,
yeah,

I don't feel qualified enough
to advance any conjecture

on how the non-trivial case
would give you algorithmic speedups,
but I guess that's the sort of promise.
And there's the idea of a speedup,
there's also the idea
of having software
that's more modular,
so having software
that for one experiment
that you can potentially reuse
to compose
with a later experiment.

Yeah,
like where that kind of lands
for me is

we can use
these nested box structures
to reflect
the articulations
of different
causal
and spatial
and temporal
systems
in sequence
and in parallel
and also proposed
cognitive mechanisms,
so not necessarily related
to the material connections
on substrate,
but related to
different kinds of interactions
amongst like attention,
memory,
all these kinds of
cognitive features.

So,

first,
there's the benefit
of the interpretability,
legibility,
modularity,
reuse,
angle,
and then
getting to that speedup,
if it's possible
to have
strong
decomposition
composition type rules,
large boxes
with a lot of stuff
in them,
whether that's
100 nestmates
in a multi-agent simulation
or 100 different
cognitive systems
in a complex
single-agent simulation,
those can be
brought down
to their most
atomic
computational rules
and possibly
certain kinds
of
calculations
will have
certain methods
that would be
like the most
effective for

this kind of
distribution
or for this
size.
So,
that gives
a unified
way
to look at
the topology
of information
flows
and calculations
in a
notational
disco-pi
framework
rather than
those connections
being kind of
semi-implicit
in the control
flow
of a Python
script
or maybe
scattered across
different files
with different
functions calling
each other,
not necessarily
in a unified
way.
For example,
a common
approach is like
to have an

agent class
and then have
them be
communicating back
and forth with
an environment
and that could
still be really
well structured
but it leads
to kind of a
fundamental
division between
what's happening
within the agent
and within
the environment
and that's
understandable
and a useful
design pattern.

However,
the stronger
direction to
kind of steer
towards would
be unifying
systems for
notation,
possibly like
disco-pi,
that could get
us like the
best of both
worlds with
well structured
interfaces and
unification across

really different
representations.

Yeah,

I think also
one important
aspect is
this idea of
separation of
concern between
syntax and
semantics where
you want the
ability to
describe your
models in a
syntactic way
that will be
reused in
different cases
with different
semantics in
mind,
different ways of
computing inference
on the same
diagram,
right?

And so with
this idea of
separating,
you have the
diagrams on one
side that
represent the
description of
your model
itself and
then you have

the functors
that describe
the computation
you do on
that model and
I think the
separation of
concern means
that well you
can compose the
syntax together
and compose these
descriptions to
form a larger
description and
then that way of
building your
model will inform
the computation
side of things
and I guess
yeah,
speedups are
sort of the
one of the
theoretical
results or
experimental
results you
could get out
of this and
I guess I
mean other
ways of thinking
about it,
I think when
you're talking
about topology

it really
feels like
there's some
connection to be
made there in
a sense that a
lot of what
people do with
these diagrams is
try and embed
them onto some
topological space
where your
computation
doesn't live in
the abstract
anymore and is
actually embedded
in a particular
space and I
I guess for
physicists this
will be like
embedded into
space-time you
know so you
will have like
these relativistic
quantum mechanics
ideas where you
have quantum
processes on a
manifold for your
space-time but I
think a similar
idea can be used
to take your
abstract description

of a generative
model and embed
it onto an actual
topological space
which is the
sort of space-like
organization of
a particular
brain or a
particular machine
right
cool yeah I
think of the
saying slow is
smooth and smooth
is fast like the
speed up sounds
like it's going to
be the pot of
gold at the end
of the rainbow but
actually the
modularity and the
legibility of the
software is what
is going to get
that infrastructure
grade long-term
templated reuse of
cognitive models and
if those take
slightly longer to
run than some
super special case
optimized situation
first off we're
only going to get
to that optimized

runtime from the
modular design and
then in the end I
think it's going to
be the
compositionality that
leads to
ecosystems of
components that are
effective and then
speed can be sort
of a secondary
consideration that's
runtime and
situation specific
but like the
benefits of
bringing category
theory to active
inference I hope we
can draw out over
the coming months
and years because
it really represents
a upgrade in how
these models are
are represented and
communicated and
they kind of bring
an aesthetic
possibility with its
own grammar that
is clearly to be
learned and also a
really strong machine
readability so I
hope that people
who view this are

excited to try
disco pi and that
we can continue to
like look back at
the models that have
been built and bring
them into these
notation systems to
demonstrate how they
can exactly recapitulate
and give reproducibility
to past examples and
I think that act
alone is going to
reveal a sort of
adjacent possible and
it would be awesome if
that could all be
happening in a
flexible environment
like disco pi.

Yeah I can't agree
more I mean I think
there's a similar
effort that's been
going on on the
machine learning side
where people have
taken category theory
to formalize existing
models and it's always
the question of okay I
mean you can do
existing stuff but
what's the new stuff
you can do with it and
I feel like yeah
active inference is

definitely a place where
you can look for
these new models where
potentially there's
something there that
you can describe in a
succinct elegant way
using categories and
show has some
experimental value in
practice and I guess
software is going to
be key to turn these
theoretical insights
into experimental
experimental value and
and I think yeah this
this ability of
category theory to make
bridges between
different topics and to
to make metaphors
become formal is also a
good way to get
inspiration from other
fields into active
inference and back so I
think I guess yeah it
would be super exciting
that the same library
gets used by quantum
natural language
processing people and
active inference that have
a priori nothing to do
with each other
cool well do you have
any last comments and

also like what
directions are you
taking it now with the
package or just in your
own work
yeah I mean I guess I
we're looking very much
for contributors
especially people who
can bring projects to
apply the library in new
directions and so please
get in touch on github or
otherwise and we'll be
very happy to to welcome
new people to the the
disco pipe contributors
and in terms of new
directions we're pushing
it in I guess yeah the
sort of education effort
is something we're trying
to invest a lot of time
in so we spend a lot of
time writing documentation
examples notebooks
hopefully will lower the
barrier to entry to
something that's quite
scary at first I mean
category theory is one of
the most intimidating
topics out there so
making it more childish
and talk about snakes
and spiders and sort of
making it fun is a part
of it and I guess yeah in

terms of developments I think there's a lot of work on category theory for probabilities and I think that has a very strong connection with functional programming like I guess there's this monad base library in high school that implements a lot of Bayesian inference in terms of monads and very elegantly in terms of category theory I think some of that could be ported to python and built into disco pi so that you could potentially yeah go beyond the sort of toy examples or you have a bunch of categorical distributions and go into something where it's yeah a very powerful system that can handle all kinds of Bayesian inference problems and in the same same sort of nice looking diagram world the future is diagrammed yeah that's I mean matrices were the maths of the 20th century diagrams will be the maths of the 21st cool thank you very much looking forward to learning more and appreciate all the work and the outreach that you

all do so yeah thanks a lot for
the invite and hope get in touch
very soon thank you bye take
care
thank you
thank you